



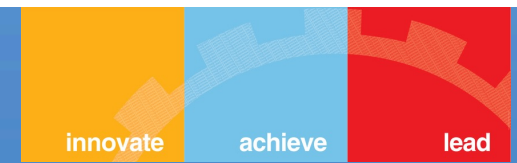
BITS Pilani

Conversational AI

ANN Search algorithm and Hybrid Retrieval

Dr Bharathi R

Adjunct Professor, CSIS WILP BITS PILANI



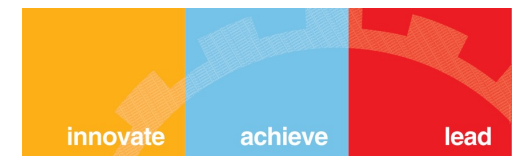
BITS Pilani



AIMLCZG521 , Conversational AI

Lecture No. 3 | Module 1

Disclaimer & Acknowledgement



DISCLAIMER

These instructional materials have been developed by the instructor for educational purposes. All content is intended solely to support the learning objectives of this course. While every effort has been made to ensure accuracy and completeness, learners are encouraged to consult original source publications for authoritative reference.

ACKNOWLEDGEMENT

These instructional materials have been developed by the instructor with due acknowledgement of the original publication authors and the broader academic community for their openly accessible resources, along with special recognition of Prof. Bhagath's Conv-AI-WILP (October 2025) presentation materials, which have been adapted and modified to support the learning objectives of this course.

1: Vector Databases

- ANN search algorithms (HNSW, IVF, PQ)
- Index structure mathematics
- Database solutions & tradeoffs
- Cost engineering

2: Sparse Retrieval

- TF-IDF fundamentals
- BM25 algorithm & mathematics
- Hybrid search rationale

3: Hybrid Systems

- Reciprocal Rank Fusion (RRF)

ANN Indexing Strategies- Revisit



1. Graph-Based (Session-2)

Example: HNSW

Build a navigable graph where nodes are vectors, edges connect similar vectors.

Pros:

- Excellent recall (>95%)
- Fast queries – $O(\log n)$
- No training needed

Cons:

- High memory usage
- Slow inserts

2. Partition-Based/Cluster based

Example: IVF

Cluster vectors into partitions, search only relevant partitions.

Pros:

- Balanced performance
- Works with compression
- Scalable

Cons:

- Needs training (k-means)
- Sensitive to nprobe

3. Compression-Based/Quantization Based

Example: PQ

Compress vectors to reduce memory, search in compressed space.

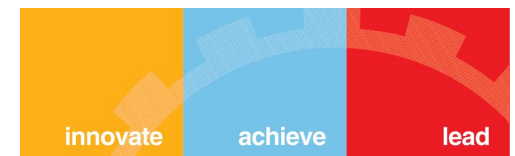
Pros:

- 8–32× less memory
- Handles billions of vectors

Cons:

- Lower recall (~90%)
- Needs training

IVF: Inverted File Index



A fast, partition-based approach to finding similar vectors at scale.

- **Partition Strategy:**

- The dataset is partitioned using *k-means clustering* into **nlist** coarse centroids. Each data point is assigned to its nearest centroid, forming an inverted list for that cluster.

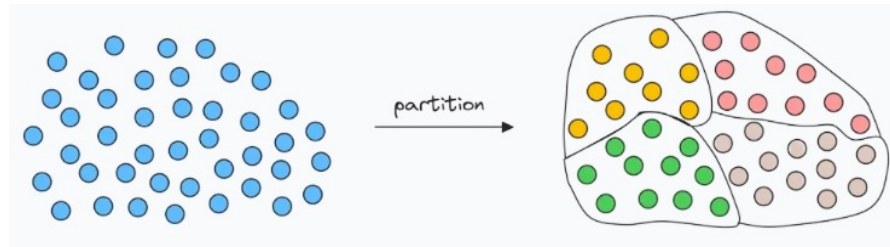
- **Search:**

- At query time, the query vector is compared to all centroids, and the top **nprobe** closest centroids are selected.
- Only the vectors in the selected clusters are searched, significantly narrowing down the search space.

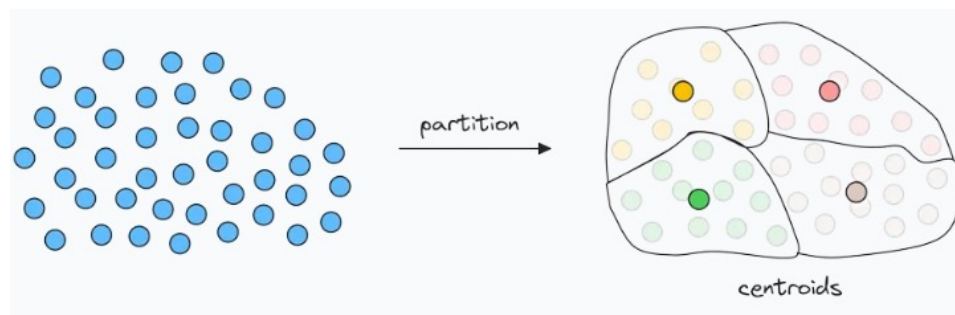
- **Index Construction:**

- A training phase precedes indexing, where representative centroids are learned from a sample of the dataset.
- Vectors are then indexed into inverted lists corresponding to their nearest centroid.

STEP 1 — PARTITION THE DATASET



Raw dataset: unorganized high-dimensional vectors



After k-means: each color = one cluster, colored dots = centroids

How it works

01 Run k-means on the dataset

Select K cluster centers (centroids) from a representative training sample. These define Voronoi cells in vector space.

02 Assign every vector to a centroid

Each data point is assigned to its nearest centroid — the one with minimum Euclidean distance.

03 Build the inverted lists

For each centroid, store the IDs of all vectors belonging to its cluster. Indexing is complete.

THE INVERTED LISTS — DATA STRUCTURE



What is stored?

Each centroid maintains an inverted list — a record of all data point IDs that belong to its cluster.



Green:

vec1, vec5, vec7, ... vec45



Orange:

vec4, vec9, vec2, ... vec41



Pink:

vec3, vec6, vec8, ... vec47

Key insight: the inverted list is built once offline. At query time it enables $O(1)$ lookup of candidate vectors — we jump directly to the relevant cluster without scanning the entire dataset.

Step 2-SEARCH PROCESS

Query Vector

Incoming query embedding

Compare to Centroids

Distance to all nlist centroids

Select nprobe Clusters

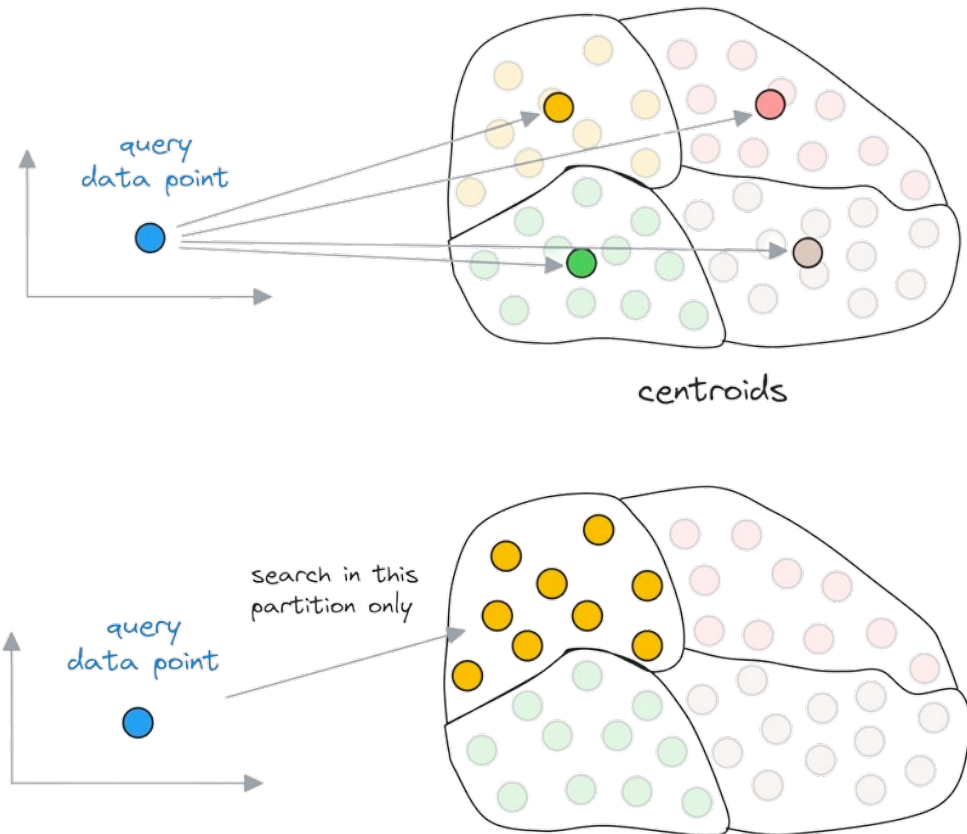
e.G nprobe=3, search top 3 clusters

Search Inverted Lists

Scan only selected cluster vectors

Return Top-K Results

Ranked by similarity score



Key Parameters

nlist

Number of k-means clusters / inverted lists.
More clusters = finer partitions.

nprobe

Clusters scanned at query time. Higher = better recall, slower speed.

NUMERICAL EXAMPLE

nlist = 3 clusters | nprobe = 1 | 2D vectors

STEP 1 — DATASET & CENTROIDS

ID	x_1	x_2	Cluster
v1	1.0	1.2	C1
v2	1.5	0.8	C1
v3	5.0	5.1	C2
v4	4.8	5.4	C2
v5	9.0	1.0	C3
v6	8.7	1.3	C3

Learned Centroids

C1

(1.25, 1.00)

C2

(4.90, 5.25)

C3

(8.85, 1.15)

STEP 2 — QUERY VECTOR

q = (5.0, 5.0)

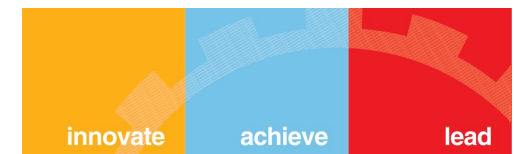
STEP 3 — EUCLIDEAN DISTANCES TO CENTROIDS

Centroid	Formula	dist ²	dist
C1 (1.25, 1.00)	$(5-1.25)^2 + (5-1)^2$	30.06	5.48
C2 (4.90, 5.25)	$(5-4.9)^2 + (5-5.25)^2$	0.07	0.26✓
C3 (8.85, 1.15)	$(5-8.85)^2 + (5-1.15)^2$	29.62	5.44

STEP 4 — SEARCH ONLY C2 (nprobe = 1)

Vector	Dist to q
v3 (5.0, 5.1)	0.10
v4 (4.8, 5.4)	0.45
✓ Result: v3 Only 2 of 6 vectors scanned	

TIME COMPLEXITY — IVF vs kNN



Variables

N	Total data points
D	Vector dimensions
K	Number of clusters (nlist)
N/K	Vectors per cluster

Total IVF complexity:

$$O(KD + \frac{ND}{K})$$

Example: N=10M, K=100
kNN → 10,000,000
IVF → 100 + 100,000 = 100,100
≈ 100× faster

Brute-Force kNN

$O(N \cdot D)$

Every query compares against all N vectors. No partitioning.

IVF — Step 1

$O(K \cdot D)$

Compare query to K centroids only. $K \ll N$.

IVF — Step 2

$O(ND/K)$

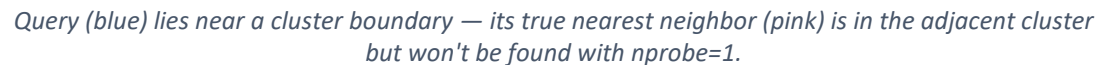
Scan 1/K of the dataset (one cluster).

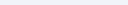
IVF Total

$O(KD + ND/K)$

With K=100: ~100× faster than brute-force kNN.

innovate **achieve** **lead**



1	Speed: 	Recall: 	Fastest — may miss boundary neighbors
nprobe	Speed: 	Recall: 	Balanced — good recall/speed tradeoff
K	Speed: 	Recall: 	Exact kNN — scans entire dataset

The recall/speed tradeoff is the fundamental contract of ANN search. Near-perfect recall at 10–100× faster than exact kNN is the goal.

PROS & CONS



✓ ADVANTAGES

Scalable

Handles billions of vectors with consistent performance

Fast Queries

Aggressive pruning eliminates irrelevant clusters instantly

Quantization-Ready

Seamlessly integrates with IVF-PQ, SQ, and other compression

Re-ranking Support

Easily pairs with re-ranking modules for improved precision

✗ LIMITATIONS

Hyperparameter Sensitivity

nlist and nprobe require careful tuning for your dataset

Training Required

k-means training phase needed before any vectors can be indexed

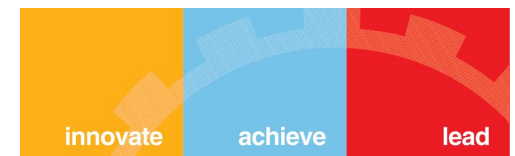
Cluster Quality Matters

Poor clustering degrades recall — outliers may fall into wrong clusters

Approximate Results

Not exact NN — recall depends on nprobe; exact search requires nprobe=nlist

USE CASES & IMPLEMENTATIONS



Visual Search

Facebook Infrastructure

Powers Facebook's visual similarity search at billion-scale, matching images by embedding proximity in real time.



Semantic Search

NLP Embeddings

Retrieves semantically similar documents or passages from large corpora using sentence-transformer embeddings.



Recommendations

Embedding-Based

Finds similar user/item embeddings for collaborative filtering and content-based recommendation engines.



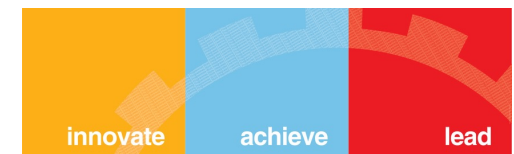
FAISS IVF-PQ

Hybrid Pipelines

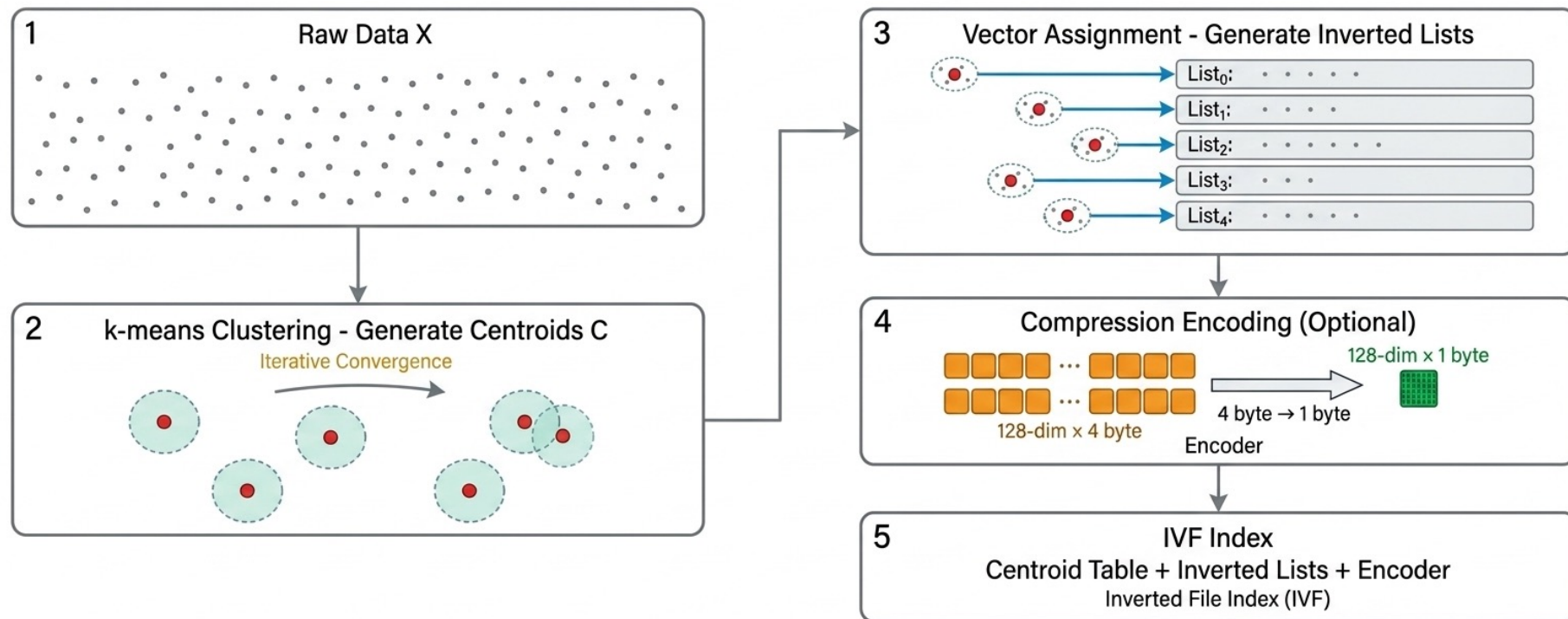
Deployed in hybrid indexing pipelines combining IVF partitioning with Product Quantization for memory efficiency.

IVF is widely deployed in FAISS IVF-PQ and hybrid vector indexing pipelines across industry.

IVF: Inverted File Index + PQ (Product Quantization)



Standard Inverted File Index (IVF) Construction Pipeline



Parameter Tuning: IVF



nlist (Number of Clusters)

$\sqrt{n}$ to $n/1000$

Too Few: Large partitions, slow search.

Too Many: Tiny partitions, many to search.

Sweet Spot: $4\sqrt{n}$ often works well.

1M vectors $\rightarrow$ nlist $\approx$ 1,000–4,000

nprobe (Clusters to Search)

1 – 100

Lower (1–5): Very fast, lower recall (~70–85%).

Higher (50–100): Slower search, higher recall (~95%+).

Tune at query time.

Typical: nprobe = 10–20

PQ Subquantizers (m)

$d/8$ to $d/4$

More subquantizers: Better accuracy, more memory.

Fewer subquantizers: More compression, lower accuracy. Must divide dimension.

768-dim $\rightarrow$ m=96 (8 per subvector)

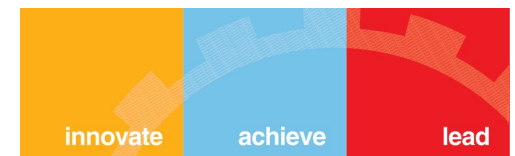
Training Time: IVF requires training k-means on sample data (typically 10K–100K vectors). Budget: nlist $\times$ 50 iterations $\times$ 10 minutes for 1M vectors.

Product Quantization

- Quantization-based methods are built on the idea of **compressing high-dimensional vectors** into compact, lossy representations, typically by mapping them to a discrete set of centroids or codes.
- These compact codes allow extremely fast **approximate distance computation (ADC)** using precomputed lookup tables, making them ideal for billion-scale vector search tasks.

- Widely used in industry for:
 - Large-scale image/video/text retrieval where space and speed are at a premium.
 - Real-time inference pipelines that need rapid ranking of semantically similar results.
 - Hybrid ANN pipelines, often paired with clustering (IVF) or re-ranking modules.
 - Scenarios that require deployment on limited-memory devices or optimized GPU environments.

Numerical Example on PQ



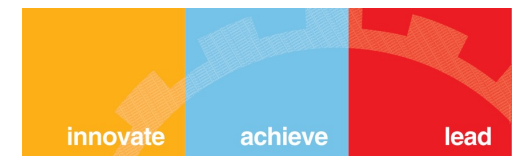
2.88 TB

To store 1 billion high-precision vectors (768 dimensions, float32), you need 2.88 terabytes of RAM.

```
1,000,000,000 vectors
  × 768 dimensions
  × 4 bytes/dimension (float32)
= 2.88 TB
```

How can we search billions of vectors without needing a warehouse of servers? → Product Quantization (PQ)!

Product Quantization (PQ)



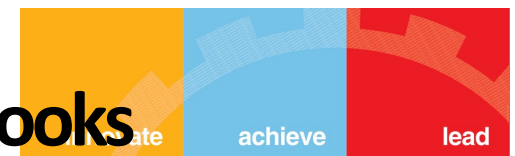
Core Concept: Divide & Conquer

Split high-dimensional vectors into smaller chunks and quantize each independently.



Each subvector is encoded independently using a trained codebook.

Why PQ? It enables massive compression (8–128×) while maintaining good search accuracy. Instead of storing 512(128* 4B) bytes per vector, you store just 4 bytes — making billion-scale search feasible in RAM.



Product Quantization (PQ) – Step 1: Train Codebooks

Step 1: Train Codebooks (k-means per Subspace)

For each subvector position, train a codebook with k centroids (typically k=256):

Codebook 1 (Sub 1)

ID: 0 → [0.2, 0.5, 0.1, ...]
ID: 1 → [0.8, 0.3, 0.6, ...]
ID: 2 → [0.1, 0.9, 0.4, ...]
...
ID: 255 → [0.4, 0.2, 0.7, ...]

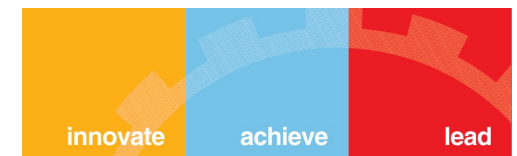
Codebook 2 (Sub 2)

ID: 0 → [0.3, 0.7, 0.2, ...]
ID: 1 → [0.6, 0.1, 0.8, ...]
ID: 2 → [0.9, 0.4, 0.3, ...]
...
ID: 255 → [0.2, 0.6, 0.5, ...]

Codebook 3 & 4...

Each subspace gets
its own codebook
trained independently
using k-means clustering

Product Quantization (PQ) – Step 2: Encode Vectors → Compact IDs



Step 2: Encode Vectors → Compact IDs

Replace each subvector with the ID of its nearest centroid in the codebook:

Original Vector (128-dim, 512 bytes):
[0.23, 0.51, 0.82, ..., 0.91, 0.34, 0.67]

↓ Encode with PQ ↓

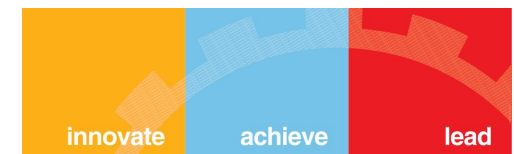
Compressed (4 IDs, 4 bytes):
[142, 73, 201, 18]
Each ID is 1 byte (0-255) → 128× compression!

Before PQ
512 B per vector



After PQ
4 B per vector

Product Quantization: Step 3 – Fast Distance Computation (ADC)



Asymmetric Distance Computation (ADC)

Step 1: Query Preparation

Query vector q arrives (full precision)
Split into subvectors: $[q_1, q_2, q_3, q_4]$
 $[0.23, 0.51, \dots] \rightarrow \text{splits} \rightarrow 4 \text{ chunks}$

Step 2: Distance Tables

Pre-compute distances from each q_i to all 256 centroids in codebook i :
 $\text{table1}[256], \text{table2}[256], \text{table3}[256], \text{table4}[256]$

Step 3: Database Vector Lookup

Each DB vector stored as 4 IDs: $[\text{ID1}, \text{ID2}, \text{ID3}, \text{ID4}]$
 $\text{distance} = \text{table1}[\text{ID1}] + \text{table2}[\text{ID2}] + \text{table3}[\text{ID3}] + \text{table4}[\text{ID4}] \quad \leftarrow \text{Just 4 lookups} + 3 \text{ additions (vs 128 multiplications)!}$

PQ Numerical Example – Step 1: The Dataset

The Dataset: We will work with 5 vectors, each with 8 dimensions.

	Dim 1	Dim 2	Dim 3	Dim 4	Dim 5	Dim 6	Dim 7	Dim 8
Vector 1	0.2	0.3	0.8	0.9	0.1	0.4	0.7	0.6
Vector 2	0.1	0.2	0.7	0.8	0.2	0.3	0.6	0.5
Vector 3	0.9	0.8	0.1	0.2	0.9	0.8	0.2	0.1
Vector 4	0.8	0.9	0.2	0.1	0.8	0.9	0.1	0.2
Vector 5	0.5	0.4	0.5	0.6	0.4	0.5	0.4	0.3

PQ Numerical Example – Step 2: Split into Subvectors



We split each 8-dimensional vector into $M=2$ subvectors, each with $d=4$ dimensions. This creates two independent "subspaces".

Vector 1:

[0.2, 0.3, 0.8, 0.9, 0.1, 0.4, 0.7, 0.6]

Subvector 1: [0.2, 0.3, 0.8, 0.9]

Subvector 2: [0.1, 0.4, 0.7, 0.6]

This same split is applied to all 5 vectors, creating 2 sets of 5 four-dimensional subvectors each.

PQ Numerical Example – Step 2: Collect Codebook Training Data



Codebook 1 (for subvector position 1) – all first subvectors:

- [0.2, 0.3, 0.8, 0.9]
- [0.1, 0.2, 0.7, 0.8]
- [0.9, 0.8, 0.1, 0.2]
- [0.8, 0.9, 0.2, 0.1]
- [0.5, 0.4, 0.5, 0.6]

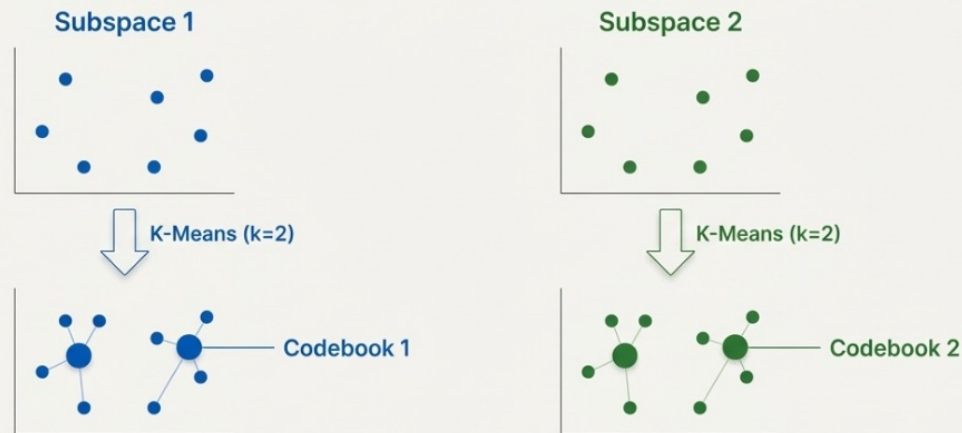
Codebook 2 (for subvector position 2) – all second subvectors:

- [0.1, 0.4, 0.7, 0.6]
- [0.2, 0.3, 0.6, 0.5]
- [0.9, 0.8, 0.2, 0.1]
- [0.8, 0.9, 0.1, 0.2]
- [0.4, 0.5, 0.4, 0.3]

Next step: Run K-Means (k=2) on each codebook's subvectors to find cluster centroids that represent typical subvector patterns.

PQ Numerical Example – Step 3: Create Codebooks with K-Means

For each subspace, we gather all the corresponding subvectors from our dataset and cluster them using K-Means. The resulting cluster centers (centroids) form our "codebook".



Each centroid is a "prototype" that represents a group of similar subvectors.

The Resulting Codebooks: Our Vector Prototypes

After running K-Means (with $k=2$) on our subvectors, we get two codebooks, each containing two 4D centroids.

Codebook 1 (for first subvectors)

Centroid 0	[0.15, 0.25, 0.75, 0.85]
Centroid 1	[0.85, 0.85, 0.15, 0.15]

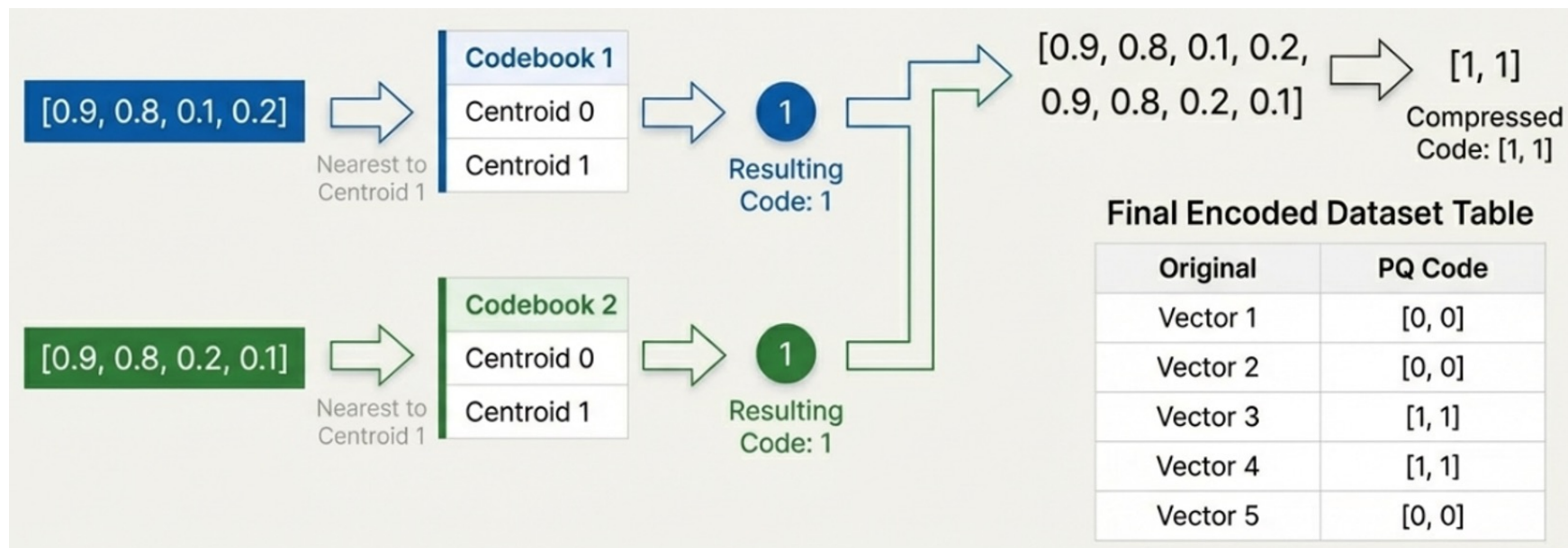
Codebook 2 (for second subvectors)

Centroid 0	[0.15, 0.35, 0.65, 0.55]
Centroid 1	[0.85, 0.85, 0.15, 0.15]

PQ Numerical Example – Step 4: Encode Vectors Using the Codebooks

For each vector, replace its subvectors with the index (ID) of the nearest centroid from the corresponding codebook.

Encoding Vector 3: [0.9, 0.8, 0.1, 0.2, 0.9, 0.8, 0.2, 0.1]



PQ Numerical Example – Step 5: Memory Savings



The Result: From 160 Bytes down to 10 Bytes

Original Storage

5 vectors $\times$ 8 dimensions
 $\times$ 4 bytes/dim (float32)

Total: 160 bytes



PQ Storage

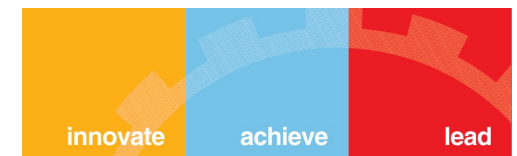
5 vectors $\times$ 2 codes $\times$ 1 byte/code
= 10 bytes (data)

2 codebooks $\times$ 2 centroids $\times$ 4 dims $\times$ 4 bytes
= 64 bytes (codebooks)

Total: 74 bytes

Key Insight: The codebook storage is a one-time, fixed cost. For millions or billions of vectors, this overhead becomes negligible, and the compression ratio approaches 32 $\times$ (from 32 bytes per vector to 1 byte per code).
Reference: Jégou et al. (2011), "Product Quantization for Nearest Neighbor Search."

PQ Numerical Example – Step 6: Similarity Search



The Challenge

A new query vector arrives. We need to find the most similar vectors in our compressed database.

Query Vector:

[0.2, 0.25, 0.75, 0.85, 0.15,
0.35, 0.65, 0.55]

The Naive Approach (What we DON'T do)

1. Decompress every vector back to its 8D approximation.
2. Compute the distance between the query and each decompressed vector.
3. This is slow and defeats the purpose of compression.

The Elegant Solution: Asymmetric Distance Computation (ADC)

We calculate distances using only the compressed codes and the codebooks, never reconstructing the full database vectors. This gives us the speed of compression with near-exact distance estimates.

PQ Numerical Example – Step 6a: Compute Distance Tables



Query vector: [0.2, 0.25, 0.75, 0.85, 0.15, 0.35, 0.65, 0.55]

Codebook 1	
Centroid 0	[0.15, 0.25, 0.75, 0.85]
Centroid 1	[0.85, 0.85, 0.15, 0.15]

Codebook 2	
Centroid 0	[0.15, 0.35, 0.65, 0.55]
Centroid 1	[0.85, 0.85, 0.15, 0.15]

For each codebook, compute the distance from the query subvector to all centroids:

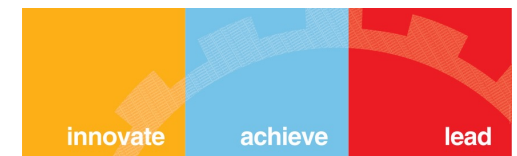
Distance Table 1:

```
dist(query_sub1, centroid_0) = 0.05
dist(query_sub1, centroid_1) = 1.17
```

Distance Table 2:

```
dist(query_sub2, centroid_0) = 0.00
dist(query_sub2, centroid_1) = 1.17
```

PQ Numerical Example – Step 6b: Compute Approximate Distances



For each encoded vector, look up distances using their codes (just table lookups, no decompression!):

Vector 1 [0, 0]

- $\text{dist} \approx 0.05 + 0.00 = 0.05$

Vector 4 [1, 1]

- $\text{dist} \approx 1.17 + 1.17 = 2.34$

Vector 2 [0, 0]

- $\text{dist} \approx 0.05 + 0.00 = 0.05$

Vector 5 [0, 0]

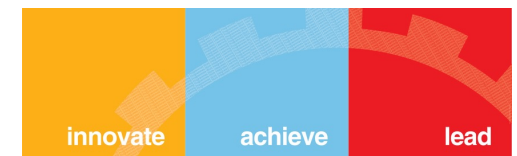
- $\text{dist} \approx 0.05 + 0.00 = 0.05$

Vector 3 [1, 1]

- $\text{dist} \approx 1.17 + 1.17 = 2.34$

Result: Vectors 1, 2, and 5 are closest to the query (distance ≈ 0.05). The key insight: we never reconstructed the full vectors! We just did table lookups.

PQ Key Parameters



M (number of subvectors):

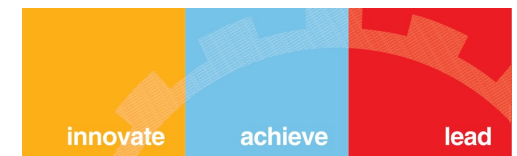
- Typical values: 8, 16, 32, 64
- More subvectors = better accuracy but more memory for codes

k (centroids per codebook):

- Typical value: 256 (fits in 1 byte)
- More centroids = better approximation but larger codebooks

Trade-off: M=8, k=256: Each vector uses 8 bytes (32× compression from float32). M=64, k=256: Each vector uses 64 bytes (4× compression, better accuracy).

Product Quantization (PQ) – Why It Is Powerful



Massive Compression

Reduce memory by 32–128× while maintaining good accuracy

Fast Search

Distance = simple table lookups instead of expensive dot products

Scalability

Store billions of vectors in RAM that would otherwise require terabytes

Combinable

Often combined with IVF for even better performance (IVFPQ)

When to Use PQ:

- Billion-scale datasets: When RAM is the bottleneck
- With IVF: IVF+PQ is the standard for massive scale
- Budget constraints: Reduce infrastructure costs 10×
- Acceptable recall loss: 90–95% recall is sufficient

BITS Pilani, Pilani Campus

Complexity Comparison – ANN Algorithms

n = vectors in database | d = vector dimension | k = clusters (IVF) |
 m = compressed dimension (PQ) | i = k-means iterations

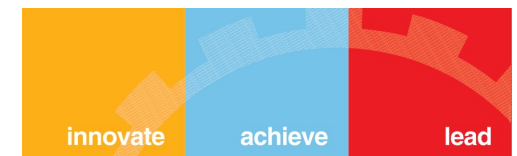
Algorithm	Search Time	Memory	Build Time	Recall
Linear Scan	$O(n \cdot d)$	$O(n \cdot d)$	$O(1)$	100%
HNSW	$O(\log n)$	$O(n \cdot d)$	$O(n \log n)$	95–99%
IVF	$O((n/k) \cdot d)$	$O(n \cdot d)$	$O(n \cdot d \cdot i)$	90–95%
PQ	$O(n \cdot m)$	$O(n \cdot m)$	$O(n \cdot d \cdot i)$	70–85%
IVF+PQ	$O((n/k) \cdot m)$	$O(n \cdot m)$	$O(n \cdot d \cdot i)$	85–92%
HNSW+PQ	$O(\log n)$	$O(n \cdot m)$	$O(n \log n)$	90–96%

Key Insight: HNSW trades memory for speed. IVF+PQ trades accuracy for compression. HNSW+PQ is the sweet spot for production.

Production Ready ANN Libraries: **FAISS**, **ScaNN**, and **ANNOY**

II. Sparse Retrieval

From Dense to Sparse: Complementary Approaches



✓ What We've Learned So Far

- Transformer encoders create rich, contextual embeddings
- Self-attention captures relationships between words
- Different pooling strategies extract sentence-level representations
- We can search billions of vectors efficiently using ANN algorithms (HNSW, IVF, PQ)

✗ But Vector Search Has Limitations...

- Struggles with exact keyword matching (e.g., "ERROR code 500")
- Misses rare/unique terms not in training data (IDs, codes)
- Can't handle out-of-vocabulary words perfectly

Next: Sparse Retrieval Methods (BM25) that complement dense search for hybrid systems

BITS Pilani, Pilani Campus

Dense Vector Search Has Limitations



Dense embeddings are semantic — they capture meaning and context beautifully. However, they often fail at keyword-exact tasks.

Limitation 1: Exact Keyword Matching

Dense models focus on meaning, not exact tokens. Numbers, error codes, and product codes may not match correctly.

Limitation 2: Rare / Unique Terms

Dense models don't index words directly. Terms unseen during training get poor embeddings (e.g., Part ID XR55-QW7).

Limitation 3: Out-of-Vocabulary (OOV) Words

Unknown words are broken into sub-tokens, leading to bad embeddings and poor ranking. BM25 handles exact string matching perfectly.

Solution: BM25 (sparse retrieval) handles all three cases perfectly!

Summary: Why We Need Sparse + Dense Together



Dense Models (BERT Embeddings):

- ✓ Great for semantic meaning
- ✓ Understand language context
- ✓ Good for long queries
- ✓ Powerful for Q&A, RAG

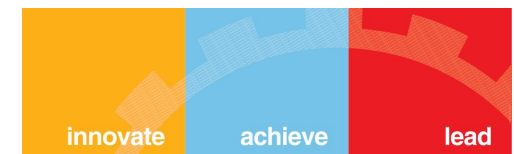
Sparse Models (BM25):

- ✓ Great for exact matching
- ✓ IDs, numbers, rare words
- ✓ Log messages, code
- ✓ Unseen terms, spelling variations

Hybrid Search = Best of Both Worlds

Dense search + BM25 improves accuracy significantly.
Dense finds semantically similar content, Sparse ensures exact matches aren't missed.

TF-IDF: Foundation of Sparse Retrieval



Term Frequency – Inverse Document Frequency: A statistical measure of word importance in documents

Term Frequency (TF)

Raw count:

$TF(t, d) = \text{count of term } t \text{ in document } d$

Normalized (common):

$TF(t, d) = f_{t,d} / |d|$

$f_{t,d}$ = raw count, $|d|$ = total terms

Intuition: More occurrences = more important

But normalize by document length.

Inverse Document Frequency (IDF)

Formula:

$IDF(t) = \log(N / df_t)$

N = total documents

df_t = documents containing t

Intuition: Rare terms are more informative.

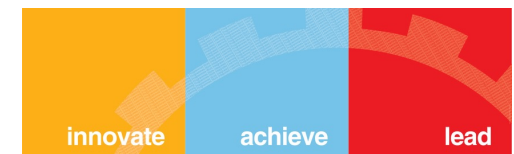
"Qdrant" → high IDF

"the" → low IDF

Combined TF-IDF Score:

$$TF-IDF(t, d) = TF(t, d) \times IDF(t) = (f_{t,d} / |d|) \times \log(N / df_t)$$

TF-IDF: Foundation of Sparse Retrieval – Worked Example



Corpus: 1000 documents | Document d: "machine learning is a subset of machine learning" | Query q: "machine learning"

Step 1: Calculate IDF for Each Query Term

Term "machine"

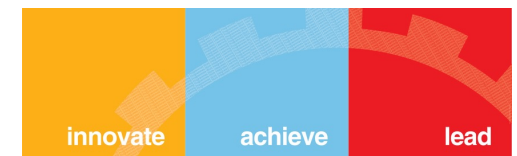
$f_{\text{machine},d} = 2$
Frequency (f): 2
 $TF = 2/8 = 0.25$
 $df \text{ (from slide)} = 300$
 $IDF = \ln(1000/300) = 1.204$
 $TF-IDF = 0.25 \times 1.204 = 0.301$

Term "learning"

Frequency (f): 2
 $TF = 2/8 = 0.25$
 $df \text{ (from slide)} = 400$
 $IDF = \ln(1000/400) = 0.916$
 $TF-IDF = 0.25 \times 0.916 = 0.229$

Document score for query "machine learning":
 $\text{Score}(q, d) = 0.301 + 0.229 = 0.530$

TF-IDF: Foundation of Sparse Retrieval – Limitations



✗ Linear TF

10 occurrences should not score 10× higher than 1. Diminishing returns are natural but not captured.

✗ No Length Normalization

Long documents score higher unfairly — they contain more words and thus more term occurrences.

✗ Bag of Words

Ignores term proximity and order. "dog bites man" and "man bites dog" get the same score.

✗ No Term Saturation

Repetition keeps adding score indefinitely. Each additional occurrence adds the same weight.

Solution: BM25 (Best Matching 25) fixes all these issues with term frequency saturation, length normalization, and a probabilistic foundation.

BM25: Best Matching 25 (Improved TF-IDF)



What BM25 Improves – BM25 is a probabilistic ranking function that addresses TF-IDF's limitations:

1. Term frequency saturation: Diminishing returns for repeated terms
2. Document length normalization: Fair scoring regardless of length
3. Probabilistic foundation: Based on probability ranking principle

BM25 Formula (Full Version):

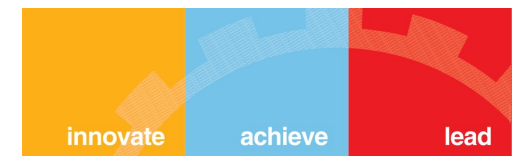
$$\text{score}(Q,D) = \sum_{i=1}^n \text{IDF}(q_i) \cdot [f(q_i,D) \cdot (k_1+1)] / [f(q_i,D) + k_1 \cdot (1-b+b \cdot |D|/\text{avgdl})]$$

Parameter	Default	Range	Effect
k_1	1.5	[1.2, 2.0]	Controls TF saturation. Higher = more emphasis on term frequency.
b	0.75	[0, 1]	Length normalization. 0 = no norm, 1 = full norm.

- $f(t,d)$: frequency of term t in document d
- $|d|$: length of the document
- avgdl : average document length in the collection
- k_1 and b : parameters that can be tuned for different datasets

Reference: Robertson & Zaragoza (2009), "The Probabilistic Relevance Framework: BM25 and Beyond."

BM25: IDF and TF Components



IDF Component (Modified)

$$\text{IDF}(q_i) = \log[(N - df_i + 0.5) / (df_i + 0.5) + 1]$$

Components:

- N = total documents
- df_i = docs containing term i
- $+0.5$ = smoothing (avoid zero)
- $+1$ = ensure positive score

Combined effect: Short docs with focused terms rank high, long docs need more evidence.

TF Component (Saturating)

$$\text{TF} = f \cdot (k_1 + 1) / (f + k_1 \cdot \text{norm})$$
$$\text{norm} = 1 - b + b \cdot (|D| / \text{avgdl})$$

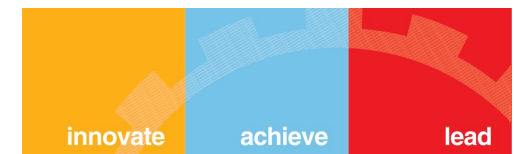
Key Innovation: Saturation function

As $f \rightarrow \infty$, $\text{TF} \rightarrow (k_1 + 1) / (k_1 \cdot \text{norm})$
Bounded, not linear!

k_1 controls saturation: With $k_1=1.5$, going from 1→2 occurrences helps more than 10→11.

b controls length penalty: $b=0.75$ means longer docs get penalized, but not fully.

BM25: Including Saturation – Intuition



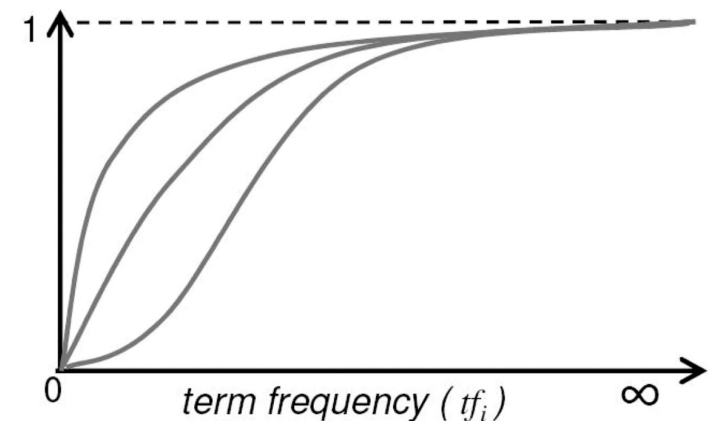
Saturation is the idea that words should get lesser weightage or value as they occur more and more in a document.

Example: Consider the document:

"Cats and dogs are pet animals though I prefer dogs. Dogs obey our commands, can be trained easily, and play with us all the time."

What if the document keeps repeating the term "dogs" over and over again? As the term frequency increases, BM25 gives lesser and lesser additional weightage to each new occurrence — the curve flattens (saturates).

Saturation Curve:



This is the key advantage of BM25 over TF-IDF: repetition keeps adding score in TF-IDF, but BM25 applies diminishing returns. Source: Robertson et al. (foundations_bm25_review.pdf)

BM25: Step-by-Step Example



Scenario:

Corpus: 1000 documents, average length: 50 terms

Query: "machine learning"

Document D: "Machine learning is a subset of machine learning and deep learning" (12 terms)

Step 1: Calculate IDF for Each Query Term

Term "machine": appears in 300 documents

$$\begin{aligned} \text{IDF} &= \log[(1000 - 300 + 0.5) / (300 + 0.5) + 1] \\ \text{IDF} &= \log[700.5 / 300.5 + 1] \\ \text{IDF} &= \log[2.331 + 1] = \log[3.331] \\ &= 1.203 \end{aligned}$$

$$\text{IDF}(\text{machine}) = 1.203$$

Term "learning": appears in 400 documents

$$\begin{aligned} \text{IDF} &= \log[(1000 - 400 + 0.5) / (400 + 0.5) + 1] \\ \text{IDF} &= \log[600.5 / 400.5 + 1] \\ \text{IDF} &= \log[2.499] = 0.916 \end{aligned}$$

$$\text{IDF}(\text{learning}) = 0.916$$

BM25: Step-by-Step Example (Steps 2 & 3)



Step 2: TF Component for "machine" ($f=2$, $k_1=1.5$, $b=0.75$)

$$\begin{aligned}\text{norm} &= 1 - 0.75 + 0.75 \times (12/50) \\ \text{norm} &= 0.25 + 0.18 = 0.43\end{aligned}$$

$$\begin{aligned}\text{TF} &= [2 \times (1.5 + 1)] / [2 + 1.5 \times 0.43] \\ \text{TF} &= 5.0 / 2.645 = 1.890\end{aligned}$$

TF(machine) = 1.890

For "learning" ($f=3$, same $\text{norm}=0.43$)

$$\begin{aligned}\text{TF} &= [3 \times 2.5] / [3 + 0.645] \\ \text{TF} &= 7.5 / 3.645 = 2.058\end{aligned}$$

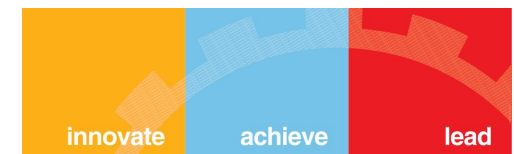
TF(learning) = 2.058

Step 3: Combine Scores

$$\begin{aligned}\text{BM25}(Q, D) &= \text{IDF}(\text{machine}) \times \text{TF}(\text{machine}) + \text{IDF}(\text{learning}) \times \text{TF}(\text{learning}) \\ &= 1.203 \times 1.890 + 0.916 \times 2.058 = 2.274 + 1.885\end{aligned}$$

BM25 Score = 4.159

BM25 vs TF-IDF – Understanding the Difference



TF-IDF score (normalized by document length):

"machine": $(2/9) \times \log(1000/300) = 0.222 \times 1.20 = 0.267$

"learning": $(3/9) \times \log(1000/400) = 0.333 \times 0.916 = 0.305$

Total = 0.572

BM25 score (with saturation & length normalization):

$\text{score}(q, d) = 4.159$

Why BM25 Scores Higher: (1) Document length normalization — doc is shorter than average (12 vs 50 tokens) → less penalty. (2) TF saturation — 3 occurrences aren't 3× better than 1. (3) Probabilistic foundation — better ranking properties.

BM25 vs TF-IDF – Understanding the Difference (Detail)



- **Document Length Normalization:**

BM25 considers document is shorter than average (12 vs 50 tokens) → less length penalty. Short, focused documents rank higher.

- **Term Frequency Saturation:**

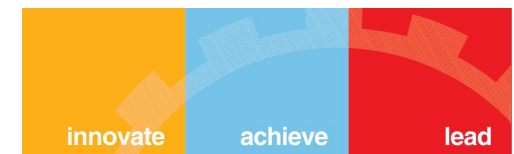
3 occurrences aren't 3× better than 1 — diminishing returns are applied. The curve flattens instead of growing linearly.

- **Probabilistic Foundation:**

Based on the probability ranking principle → better ranking properties. Documents with focused, relevant terms rank higher than verbose documents.

TF-IDF total = 0.572 vs BM25 total = 4.159

Why Hybrid Search? Dense + Sparse



✗ Dense-Only Failures – Example 1: Exact Keywords

Query: "ERROR code 500"

Doc A: "Server error 500 occurred"

Dense score: 0.72

Doc B: "Something went wrong"

Dense score: 0.85 ← WRONG RANKING!

Why? Embedding trained on general text doesn't know "500" is crucial.

✓ Sparse (BM25) Catches These – Example 1:

Query: "ERROR code 500"

Doc A: "Server error 500 occurred"

BM25 score: 8.3 (high IDF for "500")

Doc B: "Something went wrong"

BM25 score: 0.0 (no matching terms)

Correct ranking!

Why Hybrid Search? Dense + Sparse – Example

2: Unique IDs



✗ Dense-Only Failures – Example 2: Unique IDs

Query: "AIMLCZG521 schedule"

Doc A: "Course schedule for AI/ML"

Dense: 0.88 ← WRONG!

Doc B: "AIMLCZG521 timing"

Dense: 0.75

Unique terms not in training data

✓ Sparse (BM25) Catches These – Example 2:

Query: "AIMLCZG521 schedule"

Doc A: "Course schedule for AI/ML"

BM25: 3.2 ("schedule" matches)

Doc B: "AIMLCZG521 timing"

BM25: 12.5 (exact ID match!)

Correct ranking!

Dense + Sparse – Complementary Strengths



Aspect	Dense (Embeddings)	Sparse (BM25)
Semantic understanding	✓ Excellent	✗ None
Synonyms/paraphrases	✓ Handles well	✗ Misses completely
Exact keyword match	✗ Can miss	✓ Perfect
Rare/unique terms	✗ Out-of-vocabulary	✓ High IDF captures
Numbers, codes, IDs	✗ Not in training	✓ Works perfectly
Out-of-domain	⚠ Depends on training	✓ Always works

The Hybrid Solution: Use BOTH retrieval methods and combine their results! Dense finds semantically similar content, Sparse ensures exact matches aren't missed.

BITS Pilani, Pilani Campus

The Challenge: Combining Different Score Ranges



Problem: Can't just add them! Need score-agnostic fusion.

Dense retrieval gives cosine similarities $[0, 1]$ | BM25 gives unbounded scores $[0, \infty]$

Min-Max Normalization:

$$\text{score}' = (\text{score} - \text{min}) / (\text{max} - \text{min})$$

✗ Sensitive to outliers

Z-score Normalization:

$$\text{score}' = (\text{score} - \mu) / \sigma$$

✗ Assumes normal distribution

Reciprocal Rank Fusion (RRF):

$$\text{RRF}(d) = \sum 1 / (k + \text{rank}_r(d))$$

✓ No assumptions about score distributions!

Reciprocal Rank Fusion (RRF): The Math



RRF Formula

$$\text{RRF}(d) = \sum_{r \in \text{rankings}} 1 / (k + \text{rank}_r(d))$$

Where: d = document being scored | rankings = set of ranking lists (e.g., dense + sparse)
 $\text{rank}_r(d)$ = position of document d in ranking r (1, 2, 3, ...) | k = constant (typically 60)

Why $k=60$? (Empirical finding from TREC competitions)

- $k=60$ balances contribution from high and low ranks
- Without k : $1/1 = 1.0$, $1/2 = 0.5$ (too steep drop)
- With $k=60$: $1/61 = 0.0164$, $1/62 = 0.0161$ (gentler)
- Prevents first rank from dominating too much

Alternative k values:

- $k=0$: Maximum emphasis on top ranks (aggressive)
- $k=60$: Balanced (standard)
- $k=100$: More weight to lower ranks (conservative)

Reference: Cormack et al. (2009), "Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods."

Properties of RRF



1. Score-agnostic:

Only uses rank order, not raw scores. Works regardless of whether scores are $[0,1]$ or $[0,\infty]$.

2. Bounded:

Score $\in [0, \text{number of rankings} / k]$. Output is predictable and comparable across queries.

3. Symmetric:

All rankings are weighted equally. No need to specify which system is more important.

4. Robust:

Works even if one ranking is poor. The other rankings compensate, making it resilient.

5. No Training:

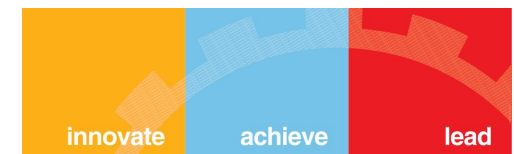
No hyperparameters to tune (except k). No labeled data needed. Plug-and-play.

5. No Training:

No hyperparameters to tune (except k). No labeled data needed. Plug-and-play.

Why RRF Works So Well: Documents that rank high in MULTIPLE systems are likely truly relevant. RRF naturally gives bonus to "consensus" documents without explicit weighting.

RRF: Example – Scenario: Query "machine learning tutorial"



Dense Retrieval Results

Rank	Doc	Cosine
1	Doc A	0.92
2	Doc C	0.89
3	Doc B	0.85
4	Doc E	0.82
5	Doc D	0.79

Sparse (BM25) Results

Rank	Doc	BM25
1	Doc B	87.3
2	Doc A	82.1
3	Doc D	79.5
4	Doc F	71.2
5	Doc C	68.9

Next: Apply RRF to combine these two ranked lists →

RRF: Example – Score Calculation (k=60)

Doc A: Dense rank: 1, Sparse rank: 2

$$\text{RRF} = 1/(60+1) + 1/(60+2) = 1/61 + 1/62 = 0.01639 + 0.01613 = 0.03252$$

Doc B: Dense rank: 3, Sparse rank: 1

$$\text{RRF} = 1/(60+3) + 1/(60+1) = 1/63 + 1/61 = 0.01587 + 0.01639 = 0.03226$$

Doc C: Dense rank: 2, Sparse rank: 5

$$\text{RRF} = 1/(60+2) + 1/(60+5) = 1/62 + 1/65 = 0.01613 + 0.01538 = 0.03151$$

Doc D: Dense rank: 5, Sparse rank: 3

$$\text{RRF} = 1/(60+5) + 1/(60+3) = 1/65 + 1/63 = 0.01538 + 0.01587 = 0.03125$$

Doc E: Dense rank: 4, Sparse rank: –

$$\text{RRF} = 1/(60+4) + 0 = 1/64 = 0.01562$$

Doc F: Dense rank: –, Sparse rank: 4

$$\text{RRF} = 0 + 1/(60+4) = 1/64 = 0.01562$$

Final Ranking

Rank	Doc	RRF Score
1	Doc A	0.03252
2	Doc B	0.03226
3	Doc C	0.03151
4	Doc D	0.03125
5	Doc E, F	0.01562

Key Insight: Doc A wins because it ranks high in BOTH systems (consensus), even though it wasn't #1 in sparse.

BITS Pilani, Pilani Campus

Performance Characteristics (Typical)

Pipeline Stage	Latency	Recall	Precision
BM25 only	20ms	70%	60%
Vector only	50ms	75%	65%
Hybrid (BM25+Vector+RRF)	70ms	90%	75%

Key Observations:

- Hybrid (BM25+Vector+RRF) achieves significantly higher recall (90%) and precision (75%) at modest latency cost (+20ms).
- BM25 alone is fast but misses semantic matches. Vector alone misses exact keyword matches.
- Combining both is the best strategy for production RAG systems.

Hybrid Search Implementation- Demo



```
# Pseudocode: Hybrid Search with RRF

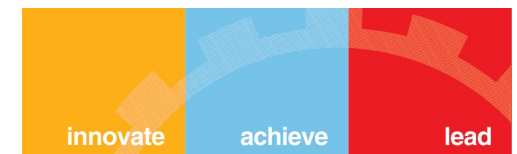
def hybrid_search(query, k=60, top_n=10):
    # Step 1: Dense Vector Search
    query_embedding = embed(query)
    dense_results = vector_db.search(query_embedding, top_n)
    # dense_results = [(doc_id, score), ...]

    # Step 2: Sparse BM25 Search
    sparse_results = bm25_index.search(query, top_n)
    # sparse_results = [(doc_id, score), ...]

    # Step 3: Reciprocal Rank Fusion
    rrf_scores = {}
    for rank, (doc_id, _) in enumerate(dense_results, 1):
        rrf_scores[doc_id] = rrf_scores.get(doc_id, 0) + 1/(k + rank)
    for rank, (doc_id, _) in enumerate(sparse_results, 1):
        rrf_scores[doc_id] = rrf_scores.get(doc_id, 0) + 1/(k + rank)

    # Step 4: Sort by RRF score
    final_results = sorted(rrf_scores, key=lambda d: rrf_scores[d], reverse=True)
    return final_results[:top_n]
```

Key Takeaways



1. Encoder Models: Bidirectional transformers create contextual embeddings through self-attention and pooling.

2. Vector Similarity: Cosine similarity (or normalized dot product) measures semantic closeness in embedding space.

3. ANN Algorithms: HNSW (graph), IVF (partitions), PQ (compression) enable fast search at scale.

4. Sparse Retrieval: BM25 improves TF-IDF with saturation and length normalization for exact keyword matching.

5. Hybrid Systems: Dense + Sparse + RRF combines semantic and lexical strengths for the best retrieval accuracy.

Formulae: Self-attention: $\text{Attention}(Q,K,V) = \text{softmax}(QK^T/Vd_k)V$ | Cosine: $\cos(A,B) = A \cdot B / (\|A\| \|B\|)$ | BM25: $f(k_1+1)/(f+k_1(1-b+b|D|/\text{avgdl}))$ | RRF: $\sum 1/(k+\text{rank})$

Resources & Further Reading



Dense Passage Retrieval (Karpukhin et al. (2020))

Foundation for modern dense retrieval systems in open-domain QA.

[ACL 2020]

HNSW (Malkov & Yashunin (2018))

Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs.

[IEEE TPAMI]

Product Quantization (Jégou et al. (2011))

Product Quantization for Nearest Neighbor Search — vector compression for large-scale search.

[IEEE TPAMI]

Sentence-BERT (Reimers & Gurevych (2019))

Sentence embeddings using Siamese BERT-Networks for semantic similarity.

[EMNLP 2019]

BM25 Review (Robertson & Zaragoza (2009))

The Probabilistic Relevance Framework: BM25 and Beyond.

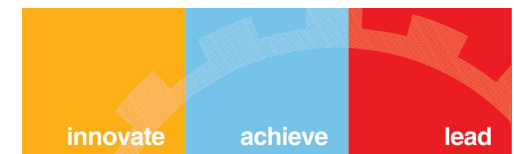
[Foundations & Trends in IR]

Reciprocal Rank Fusion (Cormack et al. (2009))

RRF outperforms Condorcet and individual rank learning methods.

[SIGIR 2009]

References



1. Dense Passage Retrieval for Open-Domain Question Answering — Karpukhin, V., et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. ACL 2020.
2. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs — Malkov, Y.A., & Yashunin, D.A. (2018). *Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs*. IEEE TPAMI.
3. Product Quantization for Nearest Neighbor Search — Jégou, H., Douze, M., & Schmid, C. (2011). *Product Quantization for Nearest Neighbor Search*. IEEE TPAMI.
4. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks — Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. EMNLP.
5. The Probabilistic Relevance Framework: BM25 and Beyond — Robertson, S., & Zaragoza, H. (2009). *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval.
6. Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods — Cormack, G.V., Clarke, C.L.A., & Buettcher, S. (2009). *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR.
7. Understanding Product Quantization: A Step-by-Step Guide — LinkedIn Article
LinkedIn Article on Product Quantization
8. <https://blog.dailydoseofds.com/p/faster-neighbor-search-using-inverted>